

Key Management —

pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

1. Key Types

Type	Algorithm	Size	Sensitivity	Zeroization
MldsPrivateKey	ML-DSA-65 (FIPS 204)	4032 bytes	Secret	ZeroizeOnDrop
MldsPublicKey	ML-DSA-65 (FIPS 204)	1952 bytes	Public	None
MldsSignature	ML-DSA-65 (FIPS 204)	3309 bytes	Public	None
MlKemPrivateKey	ML-KEM-768 (FIPS 203)	2400 bytes	Secret	ZeroizeOnDrop
MlKemPublicKey	ML-KEM-768 (FIPS 203)	1184 bytes	Public	None
MlKemSharedSecret	ML-KEM-768 (FIPS 203)	32 bytes	Secret	ZeroizeOnDrop
MlKemCiphertext	ML-KEM-768 (FIPS 203)	1088 bytes	Public	None
Hash256	SHA3-256 (FIPS 202)	32 bytes	Public	None

2. Key Generation

ML-DSA-65 keypairs

Generated via `api::generate_mldsa_keypair()`, which delegates to `pqcrypto_mldsa::mldsa65::keypair()`. The underlying implementation uses the OS-backed CSPRNG (`OsRng` via `getrandom`).

- Requires Approved state (enforced by `require_approved()` guard).
- Returns `MldsaKeyPair { public_key, private_key }`.
- The private key is wrapped in `MldsaPrivateKey` which derives `ZeroizeOnDrop`.

ML-KEM-768 keypairs

Generated via `api::generate_mlkem_keypair()`, which delegates to `pqcrypto_mlkem::mlkem768::keypair()`. The underlying implementation uses the reference C implementation of FIPS 203.

- Requires Approved state.
- Returns `MlKemKeyPair { public_key, private_key }`.
- The private key is wrapped in `MlKemPrivateKey` which derives `ZeroizeOnDrop`.

Entropy source

All key generation uses `rand::rngs::OsRng`, which is backed by the operating system's CSPRNG:

- Linux: `getrandom(2)` syscall
- macOS: `getentropy(2)` / `SecRandomCopyBytes`

The continuous RNG test (two consecutive 32-byte outputs must differ) is run at initialization.

3. Key Storage

The module does **not** persist keys to disk. All keys exist only in process memory for the duration of their Rust lifetime.

- Key material is held in `Vec<u8>` inside newtype wrappers.
- The module does not implement key export, import, or wrapping.
- Key serialization/deserialization is available via `as_bytes()` and `from_bytes()` for interoperability with the DLT layer, but the module does not control how the DLT layer handles serialized key material.

Responsibility boundary

Concern	Responsible component
Key generation	pqc_crypto_module
In-memory key protection	pqc_crypto_module (via ZeroizeOnDrop)
Key persistence to disk	DLT application layer (outside module boundary)
Key access control	DLT application layer (mTLS + ACL)
Key backup and recovery	Operator responsibility

4. Key Usage

ML-DSA-65

Operation	Function	Input	Output
Sign	api::sign_message(sk, msg)	&MldsaPrivateKey, &[u8]	MldsaSignature
Verify	api::verify_signature(pk, msg, sig)	&MldsaPublicKey, &[u8], &MldsaSignature	() or error

Keys are borrowed (&) by all operations. The module never takes ownership of caller-provided keys.

ML-KEM-768

Operation	Function	Input	Output
Encapsulate	api::mlkem_encapsulate(pk)	&MlKemPublicKey	(MlKemCiphertext, MlKemSharedSecret)
Decapsulate	api::mlkem_decapsulate(sk, ct)	&MlKemPrivateKey, &MlKemCiphertext	MlKemSharedSecret

The shared secret returned by encapsulation and decapsulation implements ZeroizeOnDrop.

Usage constraints

- All operations require Approved state.
- Invalid key sizes are rejected with `CryptoError::InvalidKey`.
- `MldsaPublicKey::from_bytes()` rejects keys that are not exactly 1952 bytes.
- `MldsaSignature::from_bytes()` rejects signatures that are not exactly 3309 bytes.

5. Key Destruction

Automatic zeroization

The following types derive `Zeroize` and `ZeroizeOnDrop` from the `zeroize` crate:

- `MlDsaPrivateKey`
- `MlKemPrivateKey`
- `MlKemSharedSecret`

When a variable of these types goes out of scope, the Rust `Drop` implementation overwrites the backing `Vec<u8>` memory with zeros before the allocator reclaims it.

Debug output redaction

Sensitive types implement custom `Debug` that prints `[REDACTED; N bytes]` instead of key material:

```
MlDsaPrivateKey([REDACTED; 4032 bytes])
MlKemPrivateKey([REDACTED; 32 bytes])
MlKemSharedSecret([REDACTED])
```

This prevents accidental logging of key material.

Limitations

- **Compiler optimizations:** The `zeroize` crate uses `write_volatile` and memory fences to resist compiler optimization of zeroization. However, no Rust-level guarantee exists against all possible compiler or OS behaviors.
- **Swap/page-out:** The module does not call `mlock()` to prevent key material from being paged to disk. This is a known gap for FIPS 140-3 Level 2+ but acceptable for Level 1.
- **Copies:** If caller code clones or copies key bytes before passing them to the module, those copies are outside the module's control.

6. Shared Secret Handling

ML-KEM-768 encapsulation produces a `MlKemSharedSecret` (32 bytes). This type:

- Implements `ZeroizeOnDrop` — cleared on drop.
- Has `as_bytes()` → `&[u8]` for the caller to read the shared secret.
- Has a redacted `Debug` implementation.
- Is not `Clone` — the caller must consume or borrow it.

The module does not use the shared secret internally. It is the caller's responsibility to use it appropriately (e.g., as input to a key derivation function).

7. Key Type Safety

All key types are distinct newtypes. The Rust type system prevents mixing:

- An `MldsPrivateKey` cannot be passed where an `MlKemPrivateKey` is expected.
- An `MldsPublicKey` cannot be passed where an `MldsPrivateKey` is expected.
- A `Hash256` cannot be confused with a key or signature.

This eliminates an entire class of key misuse bugs at compile time.